Report
Assignment 2
Introduction to Software Engineering

Group members : Anas(24p3125) and Mamoon(24p3096)

x==========x==========x

Overview Of Project

This assignment required us to fix, organize, and test an old C++ program used for calculating loan payments (EMI). The original codebase was unstable and contained three main issues: mathematical errors (integer overflow), missing input validation, and poor file organization.

Overflow Bugs : We used the safer long double data type for all loan amounts and rates. This stopped the math errors that happened with long-term loans.

Bad Input : We added validation checks to all setter functions (like setAmount). The program now instantly throws an error if a user tries to enter a negative number.

Unit Testing : We created three Google Unit Tests to prove that the code runs correctly, handles bad input, and avoids overflows.

Code Snippets

```
return (amount_*pow((1+interestPeriodic_), periodElapsed_)) -
       (payment_/interestPeriodic_)*(pow((1+interestPeriodic_), periodElapsed_)-1);
}
```

```
long double calculateLoanBalance();
long double calculatePayment();
long double calculateNumberPayments();
long double calculateLoanAmount();
long double calculateInterestRate();
// The effective interest rate, once fees have been applied
long double calculateEffectiveInterestRate();
```

```
void setInterest(long double i){
  if (i < 0 || i > 100)
      throw std::invalid_argument("Interest rate must be between 0 and 100%.");
  interest_ = i; interestPeriodic_ = i/100.0/12.0; interestSet_ = true; }
inline long double getInterest() const         { return interest_; }
inline long double getPeriodicInterest() const { return interestPeriodic_; }

void setPayment(long double P){
  if(P<0)
    throw std::invalid_argument("Amount can't be negative!");
  payment_ = P; paymentSet_ = true; }
inline long double getPayment() const { return payment_; }

void setPeriodTotal(int N){
  if(N<=0 || N> 120)
    throw std::invalid_argument("The Period must be between 0 and 10 years");
  periodTotal_ = N; periodTotalSet_ = true; }
inline int getPeriodTotal() const { return periodTotal_; }

void setPeriodElapsed(int n){
  if(n>periodTotal_ || n<=0)
    throw std::invalid_argument("The elapsed period can't be less than 0 or greater than the total period!");
  periodElapsed_ = n; periodElapsedSet_ = true; }
inline int getPeriodElapsed() const  { return periodElapsed_; }
```

```
long double LoanCalculator::calculateInterestRate()
{
  if(!amountSet_ || !paymentSet_ || !periodTotalSet_)
  {
    throw invalid_argument("Must set amount, payment, and
  }

  long double q = log10(1.0 + 1.0/periodTotal_) / log10(2
  long double monthlyInterest = pow((pow((1.0 + payment_/

  return monthlyInterest*12*100;
}

long double LoanCalculator::calculateEffectiveInterestRat
{
  if(!amountSet_ || !periodTotalSet_)
  {
    throw invalid_argument("Must set amount and total per
  }

  long double payment = calculatePayment();
  long double totalAmount = amount_ - initialPayment_;

  long double q = log10(1.0 + 1.0/periodTotal_) / log10(2
  long double monthlyInterest = pow((pow((1.0 + payment/t
```

```
inline void setOpeningFee(float fee) { openingFee_ = fee; }
inline long double getOpeningFee() const   { return openingFee_; }

inline void setOpeningPercent(float percent) { openingPercent_ = percent; }
inline long double getOpeningPercent() const       { return openingPercent_; }
```

```
inline void setAmount(long double A) {
  if(A < 0) throw std::invalid_argument("Amount can't be negative!");
  amount_ = A; amountSet_ = true;
}
```

```
private:
  long double amount_;        // loan amount
  bool amountSet_;

  long double initialPayment_;     // initial down payment

  long double interest_;          // interest rate, something like 6.75
  long double interestPeriodic_;  // this will be .0675/12
  bool interestSet_;

  long double payment_;       // payment amount
  bool paymentSet_;

  int periodTotal_;     // total payment periods
  bool periodTotalSet_;

  int periodElapsed_;   // number of elapsed payment periods
  bool periodElapsedSet_;

  // These two are used if loans charge a fee opening fee or percentage
  long double openingFee_;
  long double openingPercent_;
```

```
Running main() from ./googletest/src/gtest_main.cc
[==========] Running 3 tests from 1 test suite.
[----------] Global test environment set-up.
[----------] 3 tests from LoanCalculatorTest
[ RUN      ] LoanCalculatorTest.NormalPaymentCalculation
[       OK ] LoanCalculatorTest.NormalPaymentCalculation (0 ms)
[ RUN      ] LoanCalculatorTest.HandlesInvalidInput
[       OK ] LoanCalculatorTest.HandlesInvalidInput (0 ms)
[ RUN      ] LoanCalculatorTest.NoOverflowWithLargeTenure
[       OK ] LoanCalculatorTest.NoOverflowWithLargeTenure (0 ms)
[----------] 3 tests from LoanCalculatorTest (0 ms total)

[----------] Global test environment tear-down
[==========] 3 tests from 1 test suite ran. (0 ms total)
[  PASSED  ] 3 tests.
```

Git Commit History